A Microcontroller-based Control System for a Photomultiplier Tube Simulator

Design Laboratory, Fall 2025/2026

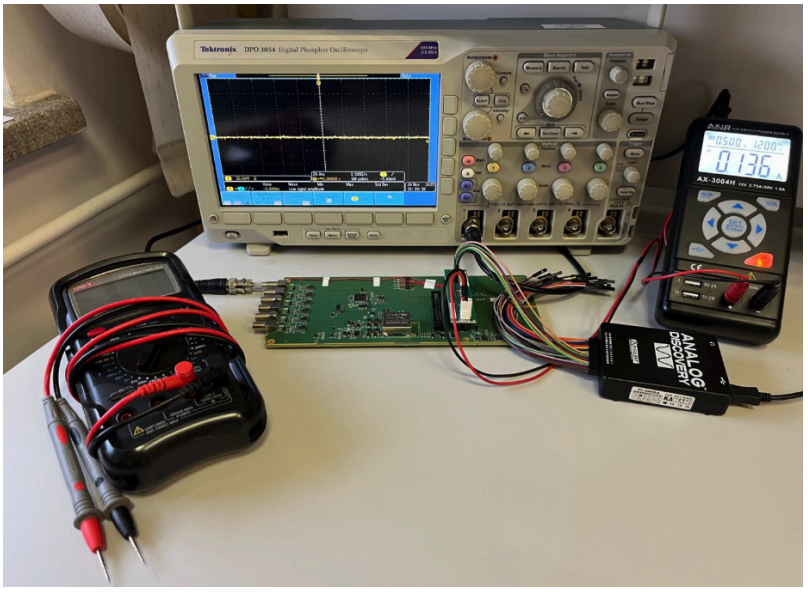
Authors: Burghardt Martyna, Bainczyk Michał
Kraków, 29.01.2026 r.

Goal of the project

The goal of the project is to develop a complete control system for a photomultiplier simulator module developed at the Warsaw University of Technology to enable easy testing and early development of the entire system without the need to use FGPAs (which are expected to be used in the final solution). An integral part of the system is a dedicated application with a graphical user interface written in Python, which allows for quick and easy configuration of the parameters of the generated pulses in real time.

Hardware

In the project we used the dedicated PCB (the "ALICE PMT Simulator" module) designed at the Warsaw University of Technology, equipped with professional signal connectors and optimized paths for high-frequency signals. The heart of the board is AD5361 digital-to-analog converter, which possesses 16 channels 14-bit resolution, enabling extremely precise amplitude settings for particle impact emulation signals. Every output connector of the board is connected to two independent channels of the converter (which can be used to emulate pulses occurring in very quick succession).



To interface transistors used to generate the impulses and the onboard AD5361 digital-to-analog converter which uses the SPI bus for communication we utilized a good old ESP32 development kit (WROOM module). ESP32 is a dual-core 32-bit microcontroller developed by Espressif Systems (China), it uses Tensilica Xtensa LX6 architecture and can be clocked up to 240 MHz. Due to low cost, a number of peripherals and ease of

development (with an Arduino support package) making prototyping very rapid, it is extensively used in hobbyist projects.

The code for ESP32

The code implements a custom control system that combines immediate command execution (single-shot mode) with continuous operation (is_repeating). Using the single_run_complete variable, the system precisely controls whether the pulse is generated only once after a parameter change or repeated cyclically.

A unique feature is the voltage_to_dac_code function, which maps the voltage to a 14-bit binary code. It uses a left-hand bit shift (<< 2) and a hardware offset of 0x2000, which is derived from the register architecture of the AD5361 chip.

To update values in the DAC registers the LDAC pin of the converter needs to be toggled. The ESP32 doesn’t control the LDAC pin of AD5361 directly, but by sending GPIO commands to the DAC's internal registers (AD5361_GPIO_FUNC_CODE). This could be done this way because the LDAC and GPIO pins of the DAC were tied together for this purpose.

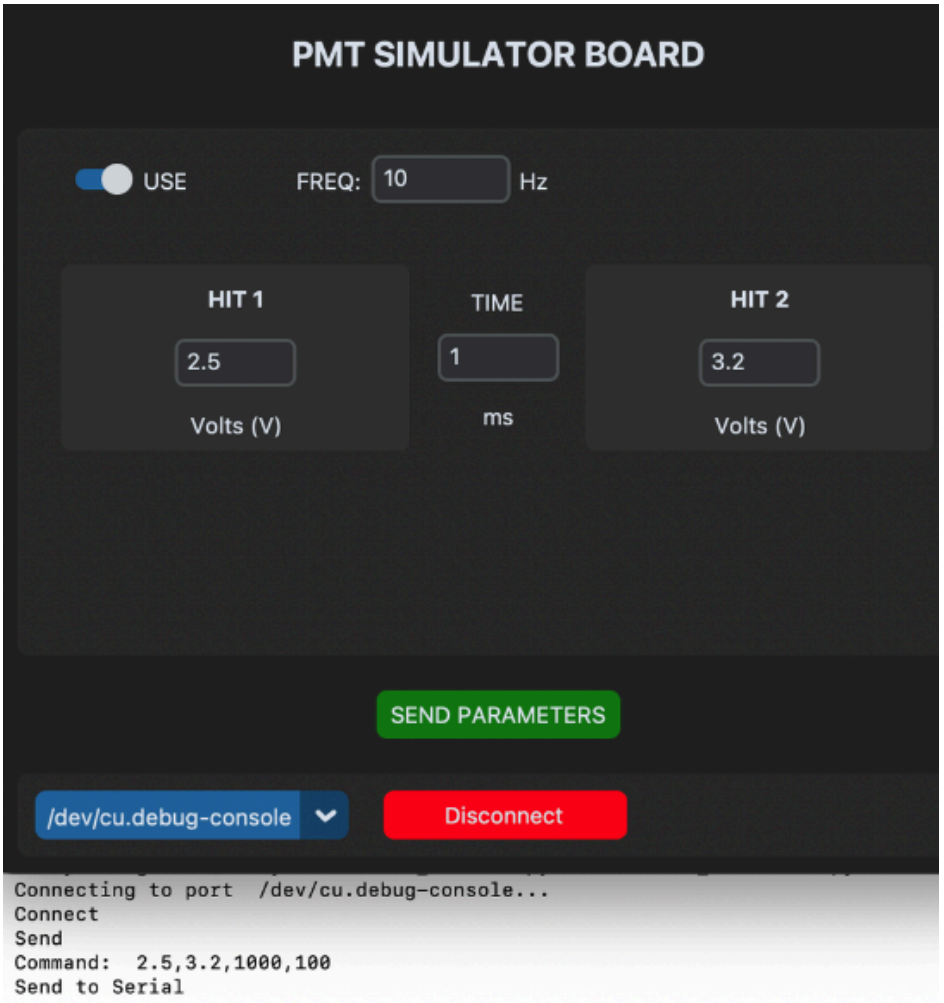
Using the sscanf function to decode input data allows for the rapid processing of complex CSV text frames. The system only updates physical parameters when all four fields (voltages and times) are correctly interpreted.

Using the delayMicroseconds() function within the trigger function allows for the generation of inter-channel delays with very fine resolution, which is crucial for emulating physical phenomena in photomultipliers.

The Python GUI app

The application was implemented in Python using the CustomTkinter library, which allowed for the creation of a graphical interface in Dark Mode. The system operates on an event-driven model, where changes to physical parameters (voltage, time) in edit fields are processed in real time after clicking the "SEND PARAMETERS" button.

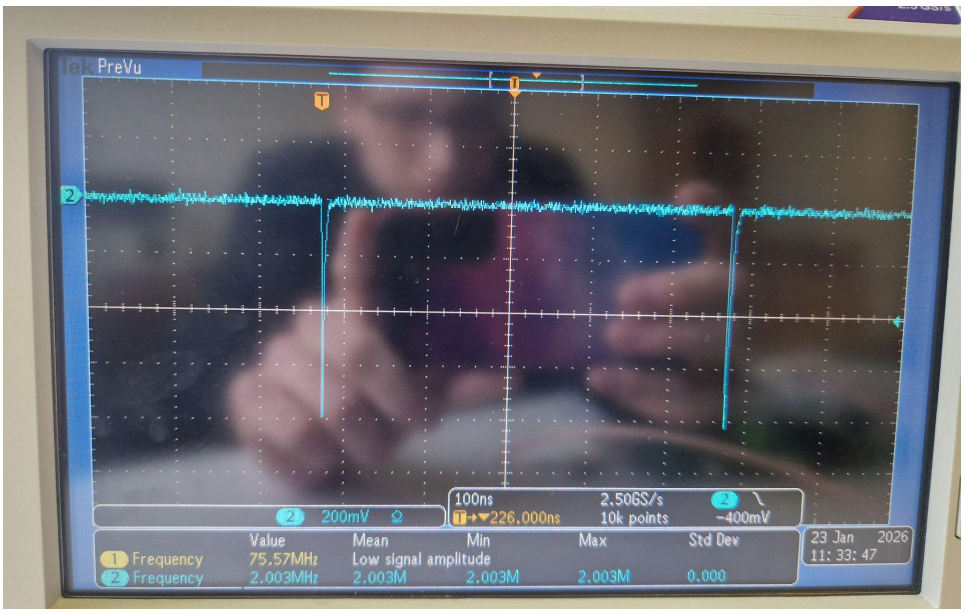
A unique element of the script is the automatic conversion of user units into process units understood by the microcontroller. Delay time given in milliseconds (ms) is converted to microseconds (µs), and frequency (Hz) to the cycle period expressed in milliseconds.



Data is packaged in a text data frame in CSV format (v1,v2,delay_us,period_ms\n). This structure ensures minimal protocol overhead and allows for easy debugging via any serial terminal. The application uses the serial.tools.list_ports library to automatically scan for available COM devices in the system, eliminating the need to manually enter device addresses. The toggle_connection function ensures safe opening and closing of data streams, and the try-except block prevents the application from crashing due to invalid characters entered or a sudden hardware disconnection.

Evaluation and conclusions

The system does work as intended with the minimal delay between subsequent pulses being 600 ns. This likely stems from the raw access time to GPIO registers on ESP32 and is a hard limit that cannot be overcome on this microcontroller. This unfortunately means that generating double pulses, with the second one beginning right after or during the first pulse is not possible but it was expected that for achieving such fine time control an FPGA would be necessary instead.


PreVu
2 200mV Ω
100ns 2.50GS/s
T→▼226.000ns 10k points
2 -400mV
Value Mean Min Max Std Dev
1 Frequency 75.57MHz Low signal amplitude
2 Frequency 2.003MHz 2.003M 2.003M 2.003M 0.000
23 Jan 2026
11: 33: 47